

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base

Manipulations
de collections

Variables,
fonctions et
collections

Classes
d'association

Opérations

Principes de
base

Spécification
d'opérations/contrats

Corps des
opérations

Etat-
Transition

Conclusion

Langage OCL : collections et éléments comportementaux

V. Hilaire

Bibliographie

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes

d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-

Transition

Conclusion

- OMG Object Constraint Language (OCL) [OMG, 2009]
- OCL(Object Constraint Language) by Example Dr. Birgit Demuth TU Dresden
- Conception par contrats avec UML / OCL - Object COnstraint Language, Gerson Sunyé LINA Université de Nantes
- Object Constraint Language (OCL) Eric Cariou
- OCL : contraintes sur métamodèles Philippe Collet

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

6 Etat-Transition

7 Conclusion

Ce que nous avons vu (IDM)

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

Un système est représenté par un modèle

De la même façon qu'un objet est une instance d'une classe.

- Le "tout est objet" était un des leit-motiv des 40 dernières années.
- De grand progrès ont été réalisés suivant ce principe.
- "Tout est modèle" est le principe de base de l'IDM.

Ce que nous avons vu (OCL)

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Les diagrammes UML peuvent être ambigus ou manquer de pouvoir d'expression pour restreindre/contraindre les éléments de modèles
- Les notes textuelles peuvent être ambiguës et ne peuvent pas être exploitées par des logiciels à des fins d'automatisation
- OCL est une réponse possible (la seule ?) pour faire face à ces questionnements en proposant la définition de contraintes attachées à des éléments de modèles pour préciser leur sémantique et leurs valeurs/configurations/comportements correct

Ce que nous allons voir

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

- Comment définir des contraintes OCL plus riches (que les contraintes simples présentées lors du dernier cours)
- Comment manipuler des collections
- Comment exprimer des contraintes sur les aspects comportementaux (méthodes, transitions, ...)

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base

Manipulations
de collections

Variables,
fonctions et
collections

Classes
d'association

Opérations

Principes de
base

Spécification
d'opérations/contrats

Corps des
opérations

Etat-
Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

6 Etat-Transition

7 Conclusion

Les contraintes simples

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

```
context: Personne
inv: self.age>0 and self.age<140
```

```
context: Personne
inv: married implies self.relative.notNull()
```

Une contrainte OCL :

- est caractérisée par un contexte (l'élément de modèle auquel elle s'applique)
- peut accéder, au sein de l'expression booléenne, aux différents éléments du contexte au travers de la notation pointée (attributs, références, ...)

Navigation par les associations

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Dans l'exemple précédent on a pu “naviguer” avec l'association liant 2 instances de Personne
- Cette association est de nature 0..1
- Dans ces cas on a donc accès à partir d'une instance de Personne à 0 ou 1 instance de classifieur qui est également de type Personne
- On peut également naviguer dans le cas d'associations 0..* ou toute autre cardinalité (autre que 1) pour cela la syntaxe n'est pas la même et on a une collection

Attention

Il faut impérativement nommer les rôles ou les associations pour naviguer.

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

- Principes de base
- Manipulations de collections
- Variables, fonctions et collections

4 Classes d'association

5 Opérations

6 Etat-Transition

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

- Principes de base
- Manipulations de collections
- Variables, fonctions et collections

4 Classes d'association

5 Opérations

6 Etat-Transition

Propriétés : collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

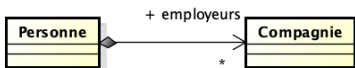
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



```
context: Personne
inv: self.employeurs->size() < 3
```

- Les types collections comme Set, OrderedSet, Bag and Sequence sont prédéfinis en OCL
- On accède aux propriétés d'une collection par l'opération
→

La navigation vue de plus prêt

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- La manipulation de collection se fait en réalité au travers de l'opération collect

```
context: Personne  
inv: self->collect(employeurs)->size() < 3
```

- Le type de retour par défaut de cette opération (prédéfinie en OCL) dans l'exemple précédent est Set(Compagnie)
- Mais l'habitude est d'écrire la version courte

```
context: Personne  
inv: self.employeurs->size() < 3
```

Types des collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

En OCL 4 types de collection d'objets

- Set : ensemble au sens mathématique, pas de doublons, pas d'ordre

```
{3,1,9,23,2}
```

- OrderedSet : idem que Set mais avec un ordre

```
{1,2,3,9,23}
```

- Bag : idem que Set mais avec une possibilité de doublons (plusieurs occurrences d'un même élément)

```
{3,1,9,3,1,23,2,1,9}
```

- Sequence : idem que Bag mais avec un ordre sur les éléments

```
{1,1,1,2,3,3,9,9,23}
```

Remarque : il est possible de transformer un type de collection en un autre type avec des opérations OCL dédiées

Retour sur les types OCL

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

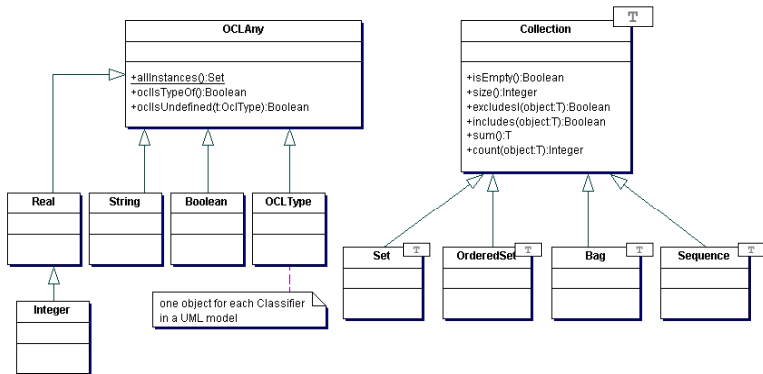
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



Règles de cohérence des types

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

Attention

OCL est un langage fortement typé

- Les expressions OCL sont vérifiées du point de vue des types manipulés
- Un type1 est conforme à un type2 si une instance de type1 peut être utilisée en remplacement d'une instance de type2
- Tous les types sont conformes avec leur super-type
- La relation d'héritage est transitive
- Un type paramétré $T(X)$ est conforme avec $T(Y)$ si X est conforme avec Y

Conformance des types

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- `oclIsTypeOf(type)` : renvoie vrai si l'objet est du type `type`
- `oclIsKindOf(type)` : renvoie vrai si l'objet est du type `type` ou un de ses sous-types
- `oclAsType(type)` : l'objet est "casté" en type `type`

Collection est le super-type de : Set, OrderedSet, Bag, Sequence

Integer est un sous-type de Real

Héritage de contraintes

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Les contraintes (OCL) sont héritées au même titre que les autres éléments d'un élément de modèle (attributs, méthodes, ...)
- Les contraintes sont attachées à l'élément de modèle défini par le contexte
- L'invariant d'un supertype doit donc être respecté par ses sous-types
- Quelquesoit le type de contrainte (invariant, pré-condition, post-condition,...) il ne peut pas être affaibli par l'héritage seulement renforcé (contrainte plus forte)

Types des collection

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

■ Collections imbriquées

```
Set {Bag{1,2,2},Bag{2}}
```

Est un exemple de collections imbriquées ou on a un ensemble (Set) composé de 2 collections (de type Bag)

- Via navigation on peut récupérer des collections ayant pour éléments d'autres collections
- Deux modes de manipulation
- Explicitement : comme une collection de collections [de collections...]
- En aplatissant pour transformer toutes les collections en une seule : l'opération flatten() permet de faire cela

```
Set {Bag{1,2,2},Bag{2}}->flatten() = Set{1,2}
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

- Principes de base
- Manipulations de collections
- Variables, fonctions et collections

4 Classes d'association

5 Opérations

6 Etat-Transition

Généralités sur les opérations sur les collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Jusque là on a plus ou moins utilisé les collections comme des attributs et/ou avec les opérations définies dans le modèle
- OCL propose des primitives pour manipuler les collections utilisables quelque soit le modèle
- Ces primitives ne sont donc pas spécifique à un contexte particulier

Opérations sur objets et collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- `size()` retourne le nombre d'éléments de la collection
- `isEmpty()` retourne vrai si la collection est vide
- `notEmpty()` retourne vrai si la collection n'est pas vide
- `includes(obj)` vrai si la collection inclut l'objet `obj`
- `excludes(obj)` vrai si la collection n'inclut pas l'objet `obj`
- `including(obj)` la collection référencée doit être cette collection en incluant l'objet `obj`
- `excluding(obj)` idem mais en excluant `obj`
- `includesAll(coll)` la collection contient tous les éléments de la collection `coll`
- `excludesAll(coll)` la collection ne contient aucun des éléments de la collection `coll`

syntaxe : `collection` → `primitive`

Opérations sur objets et collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

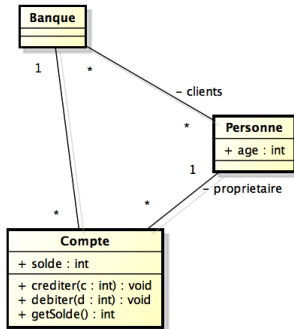
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



```
proprietaire->notEmpty():
```

il y a au moins un objet
personne associé à un
compte

```
proprietaire->size()==1
```

le nombre d'objets Personne
associés à un compte est de
1

```
banque.clients->size()>=1
```

une banque a au moins un
client

Relations ensemblistes entre collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

■ Intersection

- Primitive `intersection(collection)`
- Exemple : `col1 → intersection(col2)` renvoie l'intersection de `col1` et `col2` (les éléments en commun)
- Exemple : `(col1 → intersection(col2)) → isEmpty()` renvoie vrai si les collections `col1` et `col2` n'ont pas d'éléments en commun

■ Union

- Primitive : `union(collection)`
- Exemple : `col1 = col2 → union(col3)` construit `col1` comme l'union de `col2` et de `col3`

Opérations sur les éléments d'une collection

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

Syntaxe des primitives : 3 usages

■ `collection→primitive(expression)`

La primitive s'applique aux éléments de la collection et pour chacun d'entre eux, l'expression `expression` est vérifiée. On accède aux attributs/rerelations d'un élément directement.

■ `collection→primitive(elt:type-expression)`

On fait explicitement apparaître le type des éléments de la collection (ici `type`). On accède aux attributs/rerelations de l'élément en utilisant `let` (référence sur l'élément courant)

■ `collection→primitive(elt)`

On nomme l'élément courant sans préciser son type.

Opérations sur les éléments d'une collection

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

OCIL permet de vérifier des contraintes sur chaque éléments d'une collection ou de définir une sous-collection à partir d'une collection en fonction de certaines contraintes :

Primitives s'appliquant sur une collection col

- **select** : retourne le sous-ensemble de la collection dont les éléments respectent la contrainte spécifiée
- **reject** : idem mais ne garde que les éléments ne respectant pas la contrainte
- **collect** : retourne une collection (de taille identique) construire à partir des éléments de col. Le type des éléments contenus dans la nouvelle collection peut être différent de celui des éléments de col.
- **exists** : retourne vrai si au moins un élément de col respecte la contrainte spécifiée, faux sinon
- **forAll** : retourne vrai si tous les éléments de col respectent la contrainte

Select/Reject

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- L'idée des opérations select et reject est de sélectionner des éléments dans une collection
- La syntaxe est la suivante

```
collection -> select (elem:T | boolean expression) : collection
```

```
collection -> reject (elem:T | boolean expression) : collection
```

- Le résultat est donc une collection, potentiellement vide, des éléments qui rendent l'expression booléenne vraie pour le select et faux pour le reject

Exemple de Select/Reject

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

```
context Departement
inv: self.enseignants->select(age > 50)->notEmpty()
self . enseignants->reject(age < 23)->isEmpty()
```

```
context Departement
inv: self.enseignants->select(e| e.age > 50)->notEmpty()
```

```
context Departement
inv: self.enseignants->select(e:Enseignant| e.age > 50)->notEmpty()
```

Opérations sur les éléments d'une collection

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

Dans le contexte de la classe Banque :

```
compte->select(c | c.solde>1000)
```

Retourne une collection contenant tous les comptes dont le solde est supérieur à 1000 €

```
compte->reject(solde>1000)
```

Retourne une collection contenant tous les comptes dont le solde n'est pas supérieur à 1000 €

```
compte->collect(c:Compte|c.solde)
```

Retourne une collection contenant l'ensemble des soldes de tous les comptes

```
(compte->select(solde>1000))->collect(c|c.solde)
```

Retourne une collection contenant tous les soldes des comptes dont le solde est supérieur à 1000 €

Quelquesoit/il existe

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes

d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-

Transition

Conclusion

- L'idée est de transcrire les quantificateurs logiques $\forall$ et $\exists$ en OCL
- La syntaxe est la suivante

```
collection ->forAll(elem:T | <Boolean OCL expression>) : Boolean
```

```
collection ->exists(elem:T | <Boolean OCL expression>) : Boolean
```

- Le résultat est un booléen (et pas une collection) vraie ssi l'expression booléenne est vraie pour tous les éléments (resp un élément au moins) de collection pour forAll (resp exists)

Opérations sur les éléments d'une collection

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

```
context Banque
inv: not(clients->exists(age<18))
```

Il n'existe pas de clients de la banque dont l'âge est inférieur à 18 ans

```
context Personne
inv: Personne.allInstances()
->forAll(p1,p2|p1<>p2 implies p1.nom<>p2.nom)
```

La primitive allInstances() s'applique sur une classe (pas un objet) et retourne toutes les instances de la classe référencée (ici la classe Personne)

Il n'existe pas deux instances de la classe Personne pour lesquelles l'attribut nom a la même valeur

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

- Principes de base
- Manipulations de collections
- **Variables, fonctions et collections**

4 Classes d'association

5 Opérations

6 Etat-Transition

Variables

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

Pour faciliter l'utilisation de certains attributs ou calculs de valeurs on peut définir des variables :

- Dans une contrainte OCL let variable : type=expression1 in expression2
- Exemple :

```
context Personne
inv: let argent = compte.solde->sum() in age>=18 implies argent >0
```

- Une personne majeure doit avoir de l'argent
- sum() fait la somme de tous les objets de la collection
- Dans ce type d'expression on peut faire appel aux collections et aux opérations sur les collections

Exemple de Variables

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Pour utiliser ces variables dans toutes contraintes il faut utiliser le stéréotype def :
- Ainsi on peut définir une (ou plusieurs) variable(s) utilisable(s) au sein de plusieurs contraintes

```
context Personne
def: argent:Integer = compte.solde->sum()
```

```
context Personne
inv: age>=18 implies argent>0
```

```
context Enseignant
def : eleves () : Bag(Etudiants) = self.enseigne.etudiants
inv : self.titre = Titre::prof implies self.eleves()->forAll(each | each .estAdmis())
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base
Manipulations
de collections
Variables,
fonctions et
collections

**Classes
d'association**

Opérations

Principes de
base
Spécification
d'opérations/contrats
Corps des
opérations

Etat-
Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

6 Etat-Transition

7 Conclusion

Classes d'association

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

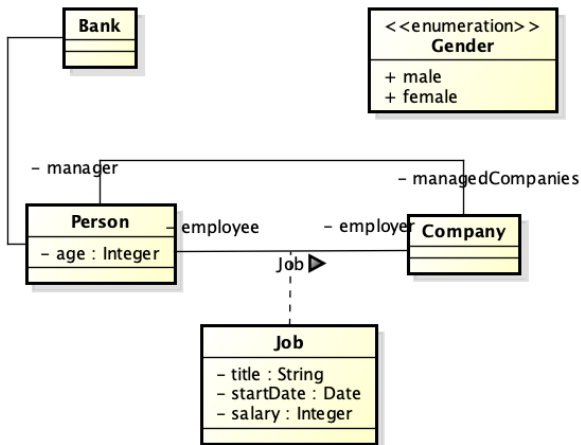
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



Classes d'association

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

- Le nom de la classe d'association est utilisé comme moyen de naviguer
- On peut accéder depuis n'importe quelle classe qui porte la relation de classe d'association
- L'expression `self.Job` peut renvoyer un Set selon les cardinalités

```
context Person  
inv: self.Job->notEmpty() implies self.Job.salary>0
```

Classes d'association

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

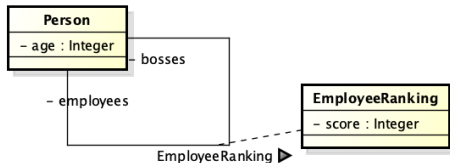
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



- Pour les associations récursives (association d'une classe vers elle-même) il faut procéder différemment
- Dans l'exemple ci-dessus on peut naviguer dans les 2 directions
- Il faut alors indiquer le nom du rôle entre crochets
- La contrainte ci-dessous exprime l'ensemble des **EmployeeRanking** appartenant aux bosses

```
context Person
inv: self.EmployeeRanking[bosses]->sum()>0
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

- Principes de base
- Spécification d'opérations/contrats
- Corps des opérations

6 Etat-Transition

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base
Manipulations
de collections
Variables,
fonctions et
collections

Classes
d'association

Opérations

Principes de
base
Spécification
d'opérations/contrats
Corps des
opérations

Etat-
Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

- Principes de base
- Spécification d'opérations/contrats
- Corps des opérations

6 Etat-Transition

Appel d'opération des classes

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Dans une contrainte OCL : accès aux attributs et objets “en lecture” sans effets de bord
- On peut également utiliser les opérations de la classe
- La contrainte est que l'opération ne doit pas avoir d'effet de bord, elle ne doit pas modifier l'état de l'objet

```
context Banque  
inv: compte->forAll(c|c.getSolde() >0)
```

getSolde() est une opération qui calcule une valeur sans modifier l'état du compte

Définition de fonctions de type query

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- L'opérateur def permet également de définir des fonctions d'interrogations du modèle (query)
- Ces opérations peuvent avoir des paramètres et type de retour

```
context Banque
def: soldesSup(val:Integer):
Set(Compte)=self.comptes->select(c|c.solde>val)
```

Propriétés : opérations de Query

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

On utilise la notation pointée

■ Opérations d'instance

```
context Personne
inv: self.age() > 0
```

■ Opérations de classe

```
context Personne
inv: self.age() < Personne.ageMaximum()
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

■ Principes de base

■ Spécification d'opérations/contrats

■ Corps des opérations

6 Etat-Transition

Spécification d'opérations

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes

d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-

Transition

Conclusion

- Une opération est définie par une signature, des pré-conditions et des post-conditions
- On peut contraindre les valeurs d'entrées et de sorties d'une opération
- Permet de dire ce que fait une opération et **non comment elle le fait**

Pre/Post-Condition

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- L'idée est de spécifier les conditions d'applicabilité et les effets d'une opération = contrat (ou boîte noire)
- On ne précise donc aucunement comment procède l'opération (ni algorithme ni impémentation)
- Ces contraintes sont attachées à une opération dans le modèle
- Ces contraintes ajoutent un contenu sémantique différent de ce que nous avons vu jusque là

Pré-conditions

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Ce qui doit être vrai avant que l'opération soit appelée
- Représenté par une expression OCL stéréotypée "precondition"

```
context: <classifier>::<operation> (<parameters>)  
  pre [<constraint name>] :  
    <Boolean OCL expression>
```

Exemple de pré-condition

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



```
context: Personne::getSalary()
pre: self.employed
```

- La contrainte ci-dessus s'applique à l'opération getSalary()
- Pour appeler cette opération (applicabilité) la personne (instance) doit être employée

Exemples de pré-condition (suite)

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

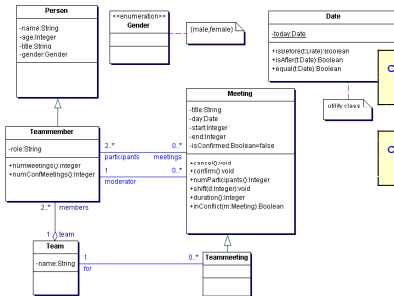
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



context: Meeting::shift(d:Integer)
pre: self.isConfirmed=false

context: Meeting::shift(d:Integer)
pre: d>0

Post-conditions

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Ce qui doit être vrai après que l'opération ait été appelée
- Représenté par une expression OCL stéréotypée "postcondition"
- Permet de spécifier le résultat d'une opération sans dire comment cette opération procède (contrat)
- Permet de représenter la valeur d'une expression avant et après l'opération
- Les principes sont les mêmes que pour les préconditions

```
context: <classifier>::<operation> (<parameters>)  
  post [constraint name] :  
    <Boolean OCL expression>
```

Exemples de postconditions

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

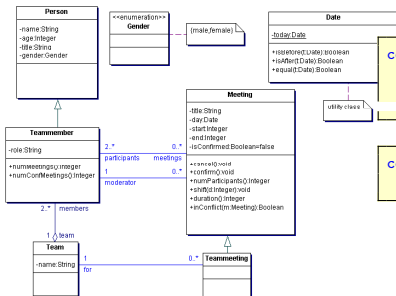
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



```
context: Meeting::duration():Integer
post: result = self.end - self.start
-- le mot clé result désigne le
   résultat
```

```
context: Meeting::confirm()
post: self.isConfirmed=true
```

Traitement des valeurs dans les éléments dynamiques

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Une opération (comme tout élément dynamique) dans un modèle peut potentiellement modifier le modèle
- Cette modification prend la forme d'un changement de valeur(s) par exemple pour des variables (attributs)
- On peut donc distinguer pour ces variables la valeur avant l'opération et la valeur après l'opération (si elle a changé)
- Une contrainte OCL permet, **uniquement dans le cadre postcondition**, d'exprimer ces 2 valeurs dans une même expression

```
context: Personne::anniversaire()  
post: age=age@pre+1
```

Exemples de postconditions

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

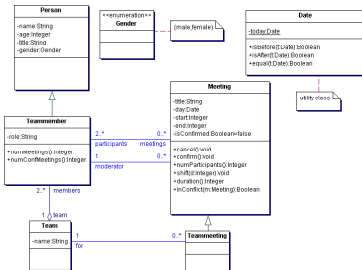
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



```
context: Meeting::shift(d:Integer)
post: start=start@pre+d
and end=end@pre+d
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

- Principes de base
- Spécification d'opérations/contrats
- **Corps des opérations**

6 Etat-Transition

Body

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

- Les contraintes pre/post peuvent être utilisées quelquesoit le type d'opération
- Pour les opérations de type requête (sans effet de bord) on peut exprimer le résultat directement
- On peut ainsi utiliser toute la puissance des opérations des collections

```
context TeamMember::getMeetingTitles():Bag(String)  
body: meetings->collect(title)
```

```
context Universite :: enseignants () : Set ( Enseignant )  
body: self.departements.enseignants->asSet()
```

Opérations sur objets et collections

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

```
context Banque::creerCompte(p:Personne):Compte
post: result.ocIsNew()
    and compte=compte@pre->including(result)
    and p.compte=p.compte@pre->including(result)
    and result.propretaire=p and result.banque=self
```

Un nouveau compte est créé.

La banque doit gérer ce nouveau compte.

Le client passé en paramètre doit posséder ce compte.

Le nouveau compte est retourné par l'opération.

L'opération `ocIsNew()` indique qu'un objet doit être créé pendant l'appel de l'opération (à utiliser en post-condition)

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base
Manipulations de collections
Variables, fonctions et collections

Classes d'association

Opérations

Principes de base
Spécification d'opérations/contrats
Corps des opérations

Etat-Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

6 Etat-Transition

7 Conclusion

Propriétés : états

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

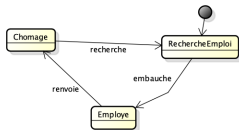
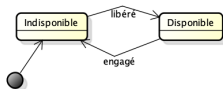
Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

On peut accéder aux états avec `oclInState()`



- L'appel prend la forme `obj.oclInState(s :oclState) : boolean`
- Par exemple

```
context: Departement::ajouter(e:
    Enseignant)
pre: e.oclInState(disponible)
```

```
context: Population
inv : travailleurs =
self.membres-> select(oclInState(
    Employe))
```

Etats imbriqués

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

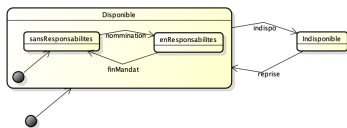
Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion



- Pour les états imbriqués on utilise la notation ::

```
context: Departement::nommerDirecteur(e:  
    Enseignant)  
pre: e.oclInState(Disponible::  
    sansResponsabilites)
```

Transitions des diagrammes états-transitions

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

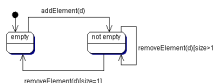
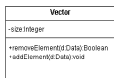
Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Une transition dans un diagramme état-transition est un élément de modèle dynamique
- Pour une transition on peut distinguer un avant le déclenchement de la transition et un après le franchissement
- La proposition OCL est d'utiliser les stéréotypes pre et post



```
context: Vector::removeElement(d:Data)
pre: oclInState(notEmpty)
post: size@pre = 1 implies oclInState(empty)
```

Plan

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base
Manipulations
de collections
Variables,
fonctions et
collections

Classes
d'association

Opérations

Principes de
base
Spécification
d'opérations/contrats
Corps des
opérations

Etat-
Transition

Conclusion

1 Introduction

2 Rappels

3 Collections

4 Classes d'association

5 Opérations

6 Etat-Transition

7 Conclusion

Bilan OCL

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- Nous avons abordé la manipulation de collections et comportements et l'expression de contraintes sur ces éléments de modèles
- Il existe bien d'autres possibilités que celles présentées (voir standard OCL)
- A partir de maintenant vous êtes en capacité d'énoncer des contraintes portant sur la plupart des diagrammes UML
- Nous verrons en TP L'utilisation d'un outil logiciel (eclipse) pour la manipulation de contraintes OCL

Conclusion

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

- OCL permet de renforcer la productivité des modèles en ajoutant :
 - du pouvoir d'expression
 - la possibilité de valider un modèle selon des contraintes

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de
base

Manipulations
de collections

Variables,
fonctions et
collections

Classes
d'association

Opérations

Principes de
base

Spécification
d'opérations/contrats

Corps des
opérations

Etat-
Transition

Conclusion

QUESTIONS ?

Application I

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

On considère un système de réunions qui impliquent des personnes. Chaque personne est caractérisé par : nom, prénom, age et titre. Une réunion possède un titre, une date, une durée, un début et une fin (valeurs entières) et peut être confirmée ou pas. Une réunion est associée à des participants et un modérateur qui sont des personnes.

Ecrire des contraintes pour :

- Pour toute réunion le nombre de participants doit être plus grand que 3
- Une réunion confirmée doit exhiber une quasi-parité de ses participants (40 % au minimum de chaque genre)
- Pour confirmer une réunion il faut que la durée soit non nulle et que le nombre de participants soit supérieur à 3
- Une personne ne peut pas participer à deux réunions la même journée

Application II

IDM

V. Hilaire

Introduction

Rappels

Collections

Principes de base

Manipulations de collections

Variables, fonctions et collections

Classes d'association

Opérations

Principes de base

Spécification d'opérations/contrats

Corps des opérations

Etat-Transition

Conclusion

On considère des véhicules à moteur, dotés d'un réservoir à carburant avec une capacité maximale et une capacité à un instant donné. Ces véhicules peuvent être soit à l'arrêt soit en marche. Un véhicule peut être associé à une personne (son conducteur) et des passagers. Un véhicule est également associé à un moteur. Les moteurs sont démarrés ou arrêtés. Tout véhicule doit être assuré pour des conducteurs, ces contrats d'assurance sont valides pour une période donnée (date de début, date de fin).

- Le nombre de passagers ne peut pas dépasser la capacité maximale du véhicule
- Le poids total des passagers (avec le poids du véhicule) ne peut pas dépasser le poids total en charge de la voiture
- Un véhicule qui roule doit être assuré pour le conducteur



OMG (2009).

OMG Object Constraint Language (OCL).

<http://www.omg.org/spec/OCL/2.3.1>.